Table des matières

1 Exercice 11 : Analyse des directions de descente **5**

1.1 Analyse théorique 5

1.2 Implémentation Matlab 5

1.3 Interprétation des résultats 6

2 Exercice 12 : Mise en place d'un code d'optimisation **7**

2.1 Simulateur 7

2.1.1 Fonction Matlab `simf0.m` 7

2.2 Optimiseur 7

2.2.1 Solution exacte du problème 7

2.2.2 Principe de la méthode de plus forte descente 8

2.2.3 Script Matlab `pfd.m` 8

2.3 Tests numériques 9

2.3.1 Nombre d'itérations nécessaires à la convergence 9

2.3.2 Tracé des itérés successifs et des courbes de niveau 10

2.3.3 Tracé de $f(x_k)$ et de $\|\nabla f(x_k)\|$ en fonction de k 10

3 Exercice 13 : Première condition de Wolfe (Diminution suffisante) **10**

3.1 Données de l'exercice 11

3.2 Première condition de Wolfe 12

3.3 Étude pour $\beta_1 = 0.5$ 12

3.3.1 Direction d_1 12

3.3.2 Direction d_2 13

3.4 Étude pour $\beta_1 = 0.1$ 13

3.4.1 Direction d_1 13

3.4.2 Direction d_2 14

3.5 Interprétation des résultats 14

4 Exercice 14 : Deuxième condition de Wolfe (Progrès suffisant) **15**

4.1 Tracée de la courbe du rapport des pentes et justification 16

4.1.1 Tracée de la courbe du rapport des pentes dans les directions de d1 et d2 16

4.1.2 Justification numérique de la linéarité (direction h) du rapport des pentes 16

4.2 Interprétation de la condition de courbure (Wolfe 2) en fonction des valeurs de β_2 17

5 Exercice 15 : Programmation de la recherche linéaire de Wolfe 19
5.1 Principe général de la recherche linéaire de Wolfe . . . 19
5.2 Programmation de la fonction `wolfe.m` . . . 19
5.3 Intégration de Wolfe dans l'algorithme de descente : fichier `gradrl.m` . . . 20
5.3.1 Script Matlab `gradrl.m` . . . 20
5.4 Résultats numériques sur la fonction f_0 . . . 21
5.4.1 Approximation du minimum . . . 21
5.4.2 Gain en nombre d'itérations . . . 22
5.5 Étude théorique de la fonction f_1 . . . 22
5.5.1 Calcul du gradient . . . 22
5.5.2 Résolution à la main . . . 23
5.5.3 Valeur de la fonction aux points critiques . . . 24
5.6 Résultats numériques sur la fonction f_1 . . . 24
5.6.1 Approximation du minimum . . . 24
5.6.2 Nombre d'itérations . . . 25
5.6.3 Nombre total d'appels au simulateur . . . 25
5.6.4 Interprétation des résultats . . . 25

6 Exercice 16 : Implémentation de la méthode de Newton locale 26
6.1 Rappel sur la méthode de Newton locale . . . 26
6.2 Données du problème . . . 26
6.3 Modification du simulateur . . . 27
6.3.1 Calcul du gradient et de la hessienne de f_0 . . . 27
6.3.2 Fonction `simf0_2.m` . . . 27
6.4 Implémentation de la méthode de Newton locale . . . 27
6.4.1 Principe de l'algorithme . . . 28
6.4.2 Fichier `newton_loc.m` . . . 28
6.5 Tests numériques sur la fonction f_0 . . . 30
6.5.1 Interprétation . . . 30
6.6 Étude théorique de la fonction f_1 . . . 30
6.6.1 Calcul du gradient . . . 30
6.6.2 Calcul de la hessienne . . . 31
6.6.3 Résolution à la main des points critiques . . . 31
6.6.4 Quels sont les minima ? . . . 32
6.6.5 Fonction `simf1_2.m` . . . 32
6.7 Tests numériques sur la fonction f_1 . . . 33
6.7.1 Résultats des tests de la méthode de Newton . . . 33
6.7.2 Interprétation . . . 36

7 Exercice 17 : Comparaison de la méthode de gradient et de la méthode de Newton locale 36
7.1 Étude théorique des fonctions . . . 36
7.1.1 Fonction f_2 . . . 36
7.1.2 Fonction f_3 . . . 37
7.2 Simulateurs Matlab . . . 37
7.2.1 Fonction `simf2_2.m` . . . 37
7.2.2 Fonction `simf3_2.m` . . . 38
7.3 Principe de la comparaison . . . 38
7.4 Tests numériques sur la fonction f_2 . . . 38
7.4.1 Carte de niveaux et trajectoires . . . 39
7.4.2 Historique de convergence . . . 39
7.4.3 Interprétation . . . 39
7.5 Tests numériques sur la fonction f_3 . . . 40
7.5.1 Carte de niveaux et trajectoires . . . 40
7.5.2 Historique de convergence . . . 41
7.5.3 Interprétation . . . 41
7.6 Synthèse . . . 41

8 Exercice 18 : Ajustement d'une gaussienne par la méthode de Gauss-Newton 42
8.1 Situation du problème . . . 42
8.2 Fonction coût à minimiser . . . 42
8.3 Écriture de la fonction sous la forme $f(a) = \frac{1}{2}\|h(a)\|^2$ . . . 43
8.4 Expression explicite de la fonction $h(a)$ . . . 43
8.4.1 Fonction `calculeh.m` . . . 43
8.5 Calcul de la matrice Jacobienne $Dh(a)$ . . . 44
8.5.1 Résidu élémentaire . . . 44
8.5.2 Dérivée par rapport à α_1 . . . 44
8.5.3 Dérivée par rapport à α_2 . . . 44
8.5.4 Dérivée par rapport à α_3 . . . 45
8.5.5 Dérivée par rapport à α_4 . . . 45
8.5.6 Forme finale de la Jacobienne . . . 45
8.6 Validation numérique de la Jacobienne . . . 46
8.6.1 Interprétation . . . 46
8.7 Implémentation Matlab du script principal . . . 48
8.7.1 Rappel : Principe de la méthode de Gauss-Newton . . . 48
8.7.2 Script Mathlab principal . . . 48
8.8 Application : Test de la fonction interpogaussienne.m . . . 50

9 Références théoriques **51**

1 Exercice 11 : Analyse des directions de descente

1.1 Analyse théorique

Soit la fonction $f(x_1, x_2) = \frac{1}{2}x_1^2 + 2x_2^2$. Pour déterminer si une direction d est une direction de descente au point $X_0 = (1, 1)$, elle doit vérifier la condition : $\nabla f(X_0)^T d < 0$. Calcul du gradient :

$$\nabla f(x_1, x_2) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{pmatrix} = \begin{pmatrix} x_1 \\ 4x_2 \end{pmatrix}$$

Au point $X_0 = (1, 1)$, on a $\nabla f(X_0) = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$. Vérification des directions :

- $d_2 = (-1, -1)$:
  $\nabla f(X_0)^T d_2 = (1)(-1) + (4)(-1) = -1 - 4 = -5 < 0$. (Direction de descente)
- $d_3 = (1, -3)$:
  $\nabla f(X_0)^T d_3 = (1)(1) + (4)(-3) = 1 - 12 = -11 < 0$. (Direction de descente)
- $d_1 = -\nabla f(X_0) = (-1, -4)$:
  $\nabla f(X_0)^T d_1 = (1)(-1) + (4)(-4) = -1 - 16 = -17 < 0$. (Direction de la plus forte pente).

1.2 Implémentation Matlab

L figure suivante permet de visualiser la décroissance de la fonction selon les trois directions pour un pas $\alpha \in [0, 1]$.

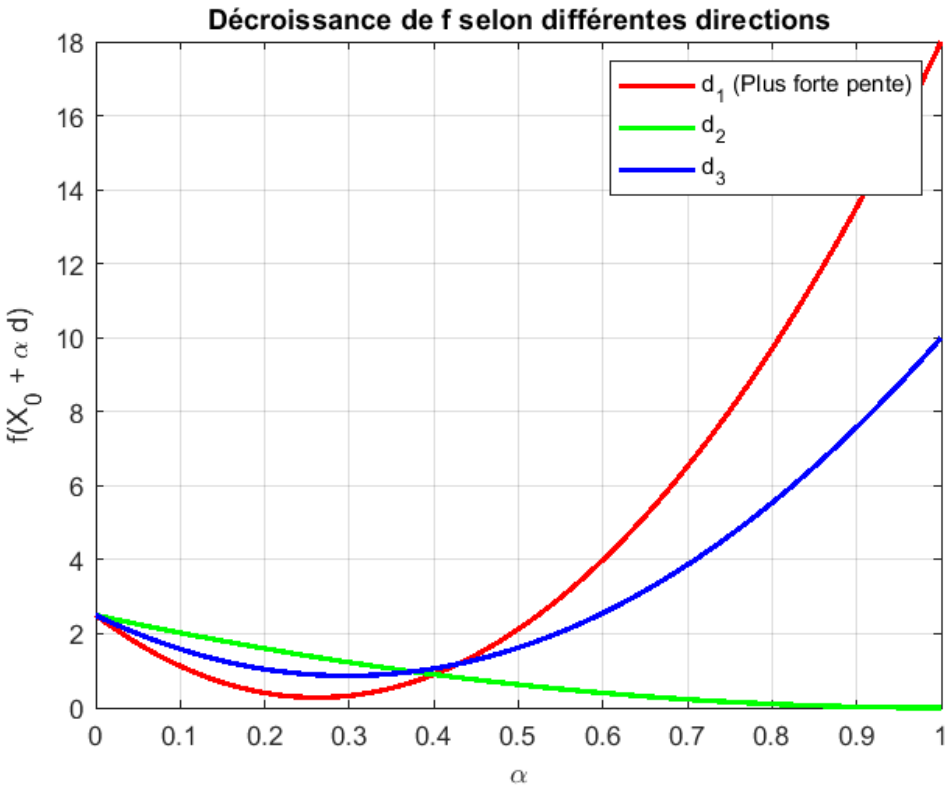


FIGURE 1 – Visualisation

1.3 Interprétation des résultats

La figure permet de voir clairement que $d_1 = -\nabla f(X_0)$ est la direction la plus efficace parmi les trois. C'est dans cette direction que la fonction diminue le plus rapidement au début, ce qui confirme bien le résultat théorique : l'opposé du gradient donne la direction de plus forte descente.

On remarque aussi que toutes les directions de descente ne sont pas équivalentes. Les directions d_2 et d_3 font bien diminuer la fonction au départ, mais moins efficacement que d_1.

De plus, la courbe associée à d_2 décroît de manière régulière, tandis que celle de d_3 commence par décroître puis remonte. Cela montre qu'une direction de descente garantit seulement une diminution locale, c'est-à-dire pour des valeurs de α relativement petites.

Ainsi, d_1 est la meilleure direction de descente au point X_0, et le choix du pas α reste essentiel pour assurer une diminution efficace de la fonction.

2 Exercice 12 : Mise en place d'un code d'optimisation

On s'intéresse au problème

$$\min_x f(x). \tag{1}$$

2.1 Simulateur

On considère la fonction définie par

$$f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2. \tag{2}$$

Son gradient est donné par

$$\nabla f_0(x) = \begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix}. \tag{3}$$

2.1.1 Fonction Matlab `simf0.m`

```
function [f,g] = simf0(x)
% x est un vecteur colonne [x1;x2]
f = 0.5*x(1)^2 + 4.5*x(2)^2;
g = [x(1);9*x(2)];
end
```

2.2 Optimiseur

2.2.1 Solution exacte du problème

La fonction f_0 est une forme quadratique strictement convexe. Son minimum est donc atteint au point où le gradient s'annule.

On résout alors

$$\nabla f_0(x) = 0 \tag{4}$$

soit

$$\begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \tag{5}$$

On obtient alors

$$x_1 = 0, \qquad x_2 = 0. \tag{6}$$

Donc la solution exacte du problème est

$$x^* = (0, 0)^T. \tag{7}$$

2.2.2 Principe de la méthode de plus forte descente

On choisit comme point de départ

$$x_0 = (9, 1)^T, \tag{8}$$

et on prend

$$\texttt{epsrel} = 10^{-8}. \tag{9}$$

À chaque itération, on applique les étapes suivantes :

— calcul de la direction de descente

$$d_k = -\nabla f(x_k), \tag{10}$$

— calcul du pas optimal α_k,

— mise à jour de l'itéré

$$x_{k+1} = x_k + \alpha_k d_k, \tag{11}$$

— arrêt lorsque le critère suivant est vérifié :

$$\|\nabla f(x_k)\|_2 \leq \texttt{epsrel}\, \|\nabla f(x_0)\|_2. \tag{12}$$

Calcul de la matrice hessienne de f_0 :

$$H = \begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix}. \tag{13}$$

Le pas optimal s'écrit alors

$$\alpha_k = \frac{\|\nabla f(x_k)\|_2^2}{\nabla f(x_k)^T H \nabla f(x_k)}. \tag{14}$$

2.2.3 Script Matlab `pfd.m`

Le script Matlab correspondant est le suivant :

```
function [x_k,k,X] = pfd(x_0,epsrel)
x_k = x_0;
k = 0;
[~,g] = simf0(x_k);
g0 = norm(g);
X = x_k;

% hessienne
```

```
H = [1 0; 0 9];

while norm(g) > epsrel*g0
% direction
d = -g;

% pas optimal
alpha = (g'*g)/(g'*H*g);

% mise a jour
x_k = x_k + alpha*d;

% recalcul
[~,g] = simf0(x_k);
% ajout
X = [X x_k];
%compteur
k = k + 1;
end
end
```

2.3 Tests numériques

2.3.1 Nombre d'itérations nécessaires à la convergence

En exécutant l'algorithme avec le point initial $x_0 = (9, 1)^T$ et `epsrel` $= 10^{-8}$, on obtient :

```
[xk,k,X] = pfd([9;1],1e-8);
disp(k)
\textbf{83}
```

L'algorithme converge donc en **83 itérations**.

2.3.2 Tracé des itérés successifs et des courbes de niveau

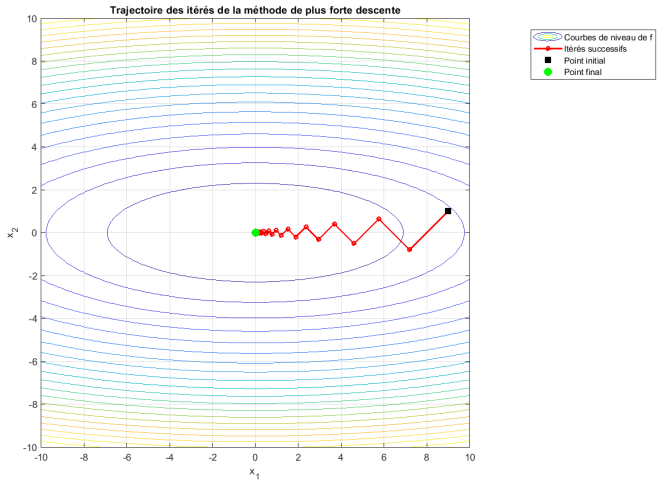


FIGURE 2 – Tracé des itérés successifs sur les courbes de niveau de f_0.

2.3.3 Tracé de $f(x_k)$ et de $\|\nabla f(x_k)\|$ en fonction de k

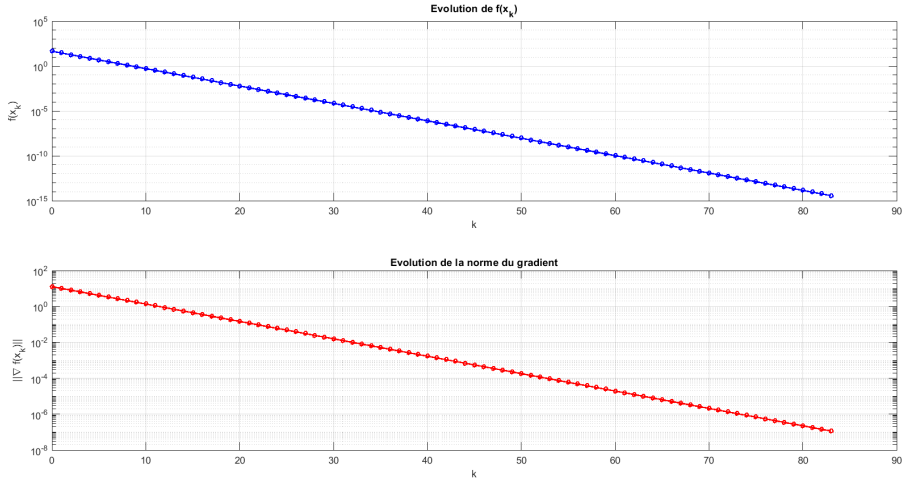


FIGURE 3 – Évolution de $f(x_k)$ et de $\|\nabla f(x_k)\|$ en fonction de k.

3 Exercice 13 : Première condition de Wolfe (Diminution suffisante)

La première condition de Wolfe, souvent appelée **condition de diminution suffisante** ou encore **règle d'Armijo**, est un critère essentiel dans les méthodes d'optimisation numérique [Arm66, Wol69, Wol71]. Elle permet de s'assurer que la réduction de la fonction objectif est proportionnelle à la longueur du pas, afin de garantir la descente, d'éviter les pas trop grands et d'assurer la stabilité de l'algorithme.

3.1 Données de l'exercice

On considère la fonction

$$f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2. \tag{15}$$

Le gradient de la fonction est donné par

$$\nabla f_0(x) = \begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix}. \tag{16}$$

Au point

$$x_0 = (10, 1)^T,$$

nous avons

$$f_0(x_0) = \frac{1}{2}(10)^2 + \frac{9}{2}(1)^2 = 50 + 4.5 = 54.5, \tag{17}$$

$$\nabla f_0(x_0) = \begin{pmatrix} 10 \\ 9 \end{pmatrix}. \tag{18}$$

On considère les deux directions suivantes :

$$D_1 = \begin{pmatrix} -10 \\ -9 \end{pmatrix}, \qquad D_2 = \begin{pmatrix} -2 \\ 1 \end{pmatrix}.$$

Après normalisation, on obtient :

$$\|D_1\| = \sqrt{(-10)^2 + (-9)^2} = \sqrt{181}, \tag{19}$$

$$d_1 = \frac{1}{\sqrt{181}} \begin{pmatrix} -10 \\ -9 \end{pmatrix}, \tag{20}$$

et

$$\|D_2\| = \sqrt{(-2)^2 + 1^2} = \sqrt{5}, \tag{21}$$

$$d_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} -2 \\ 1 \end{pmatrix}. \tag{22}$$

Les produits scalaires avec le gradient valent :

$$\nabla f_0(x_0)^T d_1 = (10, 9) \cdot \frac{1}{\sqrt{181}} \begin{pmatrix} -10 \\ -9 \end{pmatrix} = \frac{-100 - 81}{\sqrt{181}} = -\sqrt{181} \approx -13.4536, \tag{23}$$

$$\nabla f_0(x_0)^T d_2 = (10, 9) \cdot \frac{1}{\sqrt{5}} \begin{pmatrix} -2 \\ 1 \end{pmatrix} = \frac{-20 + 9}{\sqrt{5}} = -\frac{11}{\sqrt{5}} \approx -4.9193. \tag{24}$$

Comme ces deux quantités sont strictement négatives, d_1 et d_2 sont bien des directions de descente.

3.2 Première condition de Wolfe

La première condition de Wolfe s'écrit

$$f(x_k + \alpha_k d_k) \leq f(x_k) + \beta_1 \alpha_k \nabla f(x_k)^T d_k, \tag{25}$$

avec

$$0 < \beta_1 < 1.$$

Elle impose que la décroissance réelle de la fonction soit suffisante par rapport à la décroissance prédite par l'approximation linéaire.

3.3 Étude pour $\beta_1 = 0.5$

3.3.1 Direction d_1

On considère

$$\phi_1(\alpha) = f_0(x_0 + \alpha d_1).$$

Comme

$$x_0 + \alpha d_1 = \begin{pmatrix} 10 - \dfrac{10\alpha}{\sqrt{181}} \\ 1 - \dfrac{9\alpha}{\sqrt{181}} \end{pmatrix},$$

on obtient

$$\phi_1(\alpha) = \frac{1}{2}\left(10 - \frac{10\alpha}{\sqrt{181}}\right)^2 + \frac{9}{2}\left(1 - \frac{9\alpha}{\sqrt{181}}\right)^2 \tag{26}$$

$$= 54.5 - \sqrt{181}\,\alpha + \frac{829}{362}\alpha^2. \tag{27}$$

La droite tangente en $\alpha = 0$ est

$$T_1(\alpha) = f_0(x_0) + \alpha \nabla f_0(x_0)^T d_1 = 54.5 - \sqrt{181}\,\alpha. \tag{28}$$

La droite correspondant à la condition d'Armijo pour $\beta_1 = 0.5$ est

$$A_1^{(0.5)}(\alpha) = f_0(x_0) + 0.5\,\alpha \nabla f_0(x_0)^T d_1 = 54.5 - 0.5\sqrt{181}\,\alpha. \tag{29}$$

3.3.2 Direction d_2

On a

$$x_0 + \alpha d_2 = \begin{pmatrix} 10 - \dfrac{2\alpha}{\sqrt{5}} \\ 1 + \dfrac{\alpha}{\sqrt{5}} \end{pmatrix},$$

d'où

$$\phi_2(\alpha) = \frac{1}{2}\left(10 - \frac{2\alpha}{\sqrt{5}}\right)^2 + \frac{9}{2}\left(1 + \frac{\alpha}{\sqrt{5}}\right)^2 \tag{30}$$

$$= 54.5 - \frac{11}{\sqrt{5}}\alpha + 1.3\alpha^2. \tag{31}$$

La droite tangente vaut

$$T_2(\alpha) = 54.5 - \frac{11}{\sqrt{5}}\alpha. \tag{32}$$

La droite d'Armijo pour $\beta_1 = 0.5$ est

$$A_2^{(0.5)}(\alpha) = 54.5 - \frac{11}{2\sqrt{5}}\alpha. \tag{33}$$

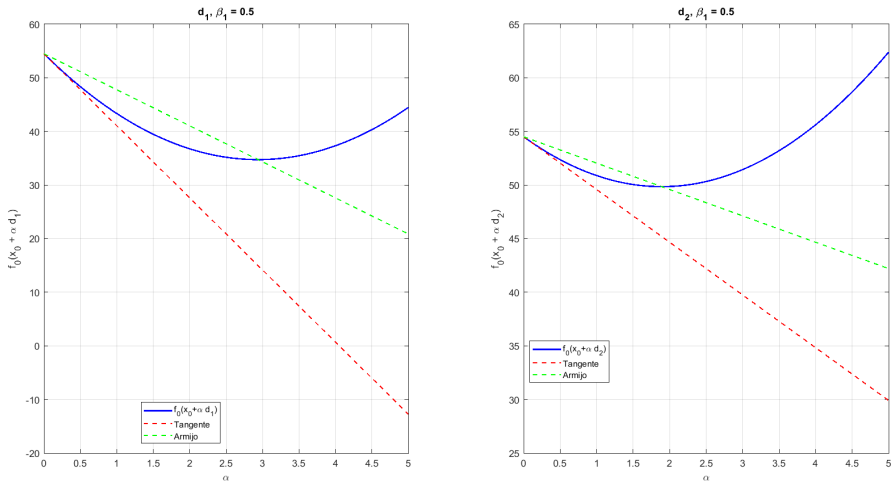


FIGURE 4 – Première condition de Wolfe pour $\beta_1 = 0.5$.

3.4 Étude pour $\beta_1 = 0.1$

3.4.1 Direction d_1

La courbe reste

$$\phi_1(\alpha) = f_0(x_0 + \alpha d_1) = 54.5 - \sqrt{181}\,\alpha + \frac{829}{362}\alpha^2. \tag{34}$$

La droite d'approximation linéaire est

$$\ell_1(\alpha) = f_0(x_0) + \alpha \nabla f_0(x_0)^T d_1 = 54.5 - \sqrt{181}\,\alpha. \tag{35}$$

La droite de Wolfe devient

$$w_1(\alpha) = f_0(x_0) + 0.1\,\alpha \nabla f_0(x_0)^T d_1 = 54.5 - 0.1\sqrt{181}\,\alpha. \tag{36}$$

3.4.2 Direction d_2

La courbe reste

$$\phi_2(\alpha) = f_0(x_0 + \alpha d_2) = 54.5 - \frac{11}{\sqrt{5}}\alpha + 1.3\alpha^2. \tag{37}$$

La droite d'approximation linéaire est

$$\ell_2(\alpha) = f_0(x_0) + \alpha \nabla f_0(x_0)^T d_2 = 54.5 - \frac{11}{\sqrt{5}}\alpha. \tag{38}$$

La droite de Wolfe devient

$$w_2(\alpha) = f_0(x_0) + 0.1\,\alpha \nabla f_0(x_0)^T d_2 = 54.5 - 0.1\frac{11}{\sqrt{5}}\alpha = 54.5 - \frac{1.1}{\sqrt{5}}\alpha. \tag{39}$$

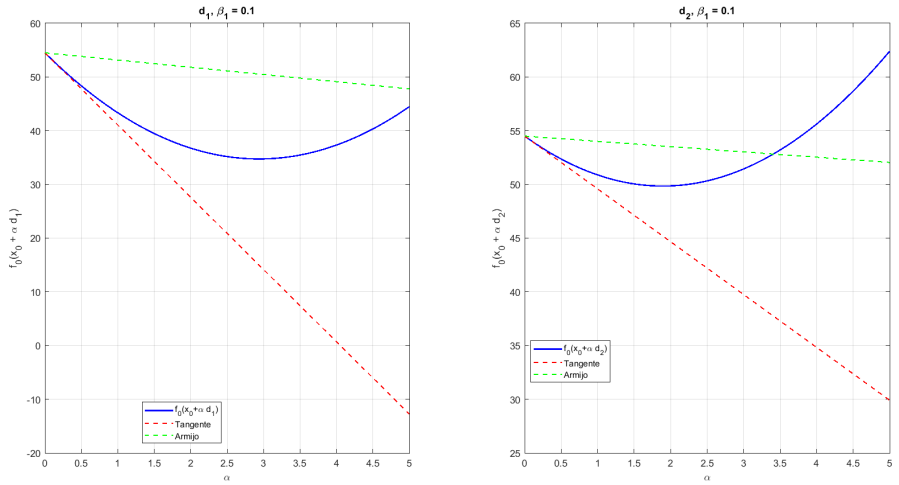


FIGURE 5 – Première condition de Wolfe pour $\beta_1 = 0.1$.

3.5 Interprétation des résultats

Les figures ci-dessus permettent de comprendre le rôle de la première condition de Wolfe :

$$f(x_0 + \alpha d) \leq f(x_0) + \beta_1 \alpha \nabla f_0(x_0)^T d.$$

La droite de Wolfe sert de ligne de comparaison :

— Si la courbe réelle de la fonction reste en dessous de cette droite, alors le pas α est acceptable ;

— Si elle passe au-dessus, cela signifie que la diminution n'est pas suffisante.

• Lorsque $\beta_1 = 0.5$, la droite de Wolfe est fortement inclinée vers le bas. Les pas acceptés diminuent donc fortement la fonction et l'ensemble des pas admissibles est plus réduit.

• Lorsque $\beta_1 = 0.1$, la droite de Wolfe est plus plate. Il devient alors plus facile pour la courbe réelle de rester en dessous et davantage de valeurs de α sont acceptées.

Ainsi, le paramètre β_1 contrôle le niveau d'exigence de la descente :

— Une petite valeur de β_1 rend la condition plus facile à satisfaire ;

— Une valeur plus grande de β_1 rend la condition plus stricte.

Remarque : Cependant, la première condition de Wolfe présente une limite importante : elle ne cherche pas le meilleur pas possible. Elle vérifie seulement si la diminution obtenue est suffisante.

Par conséquent, un pas peut encore être accepté même après avoir dépassé le minimum de la courbe, tant que la fonction reste en dessous de la droite d'Armijo. Ainsi, cette condition n'empêche pas toujours d'accepter des pas situés après le minimum comme le montre bien le cas avec $\beta_1 = 0.1$; elle rejette seulement les pas devenus vraiment trop grands, c'est-à-dire ceux pour lesquels la diminution n'est plus suffisante.

On peut donc conclure que la première condition de Wolfe garantit une décroissance acceptable de la fonction, mais elle ne garantit pas que le pas choisi soit optimal.

4 Exercice 14 : Deuxième condition de Wolfe (Progrès suffisant)

La deuxième condition de Wolfe [Wol69, Wol71] s'écrit mathématiquement :

$$\nabla f(x_k + \alpha_k d_k)^T d_k \geq \beta_2 \nabla f(x_k)^T d_k \tag{40}$$

où $0 < \beta_1 < \beta_2 < 1$. Elle peut encore s'écrire :

$$\frac{\nabla f(x_k + \alpha_k d_k)^T d_k}{\nabla f(x_k)^T d_k} \leq \beta_2 \tag{41}$$

Où :

— d_k est la direction de descente au point x_k.

— α_k est la longueur du pas.

— $\beta_2 \in [0, 1]$ est le paramètre de courbure.

Elle est encore appelée **condition de courbure**. Elle empêche l'algorithme de choisir des pas trop petits. Elle garantit que la pente au nouveau point est significativement plus grande (moins négative) que la pente initiale, forçant ainsi un déplacement vers le minimum.

4.1 Tracée de la courbe du rapport des pentes et justification

4.1.1 Tracée de la courbe du rapport des pentes dans les directions de d1 et d2

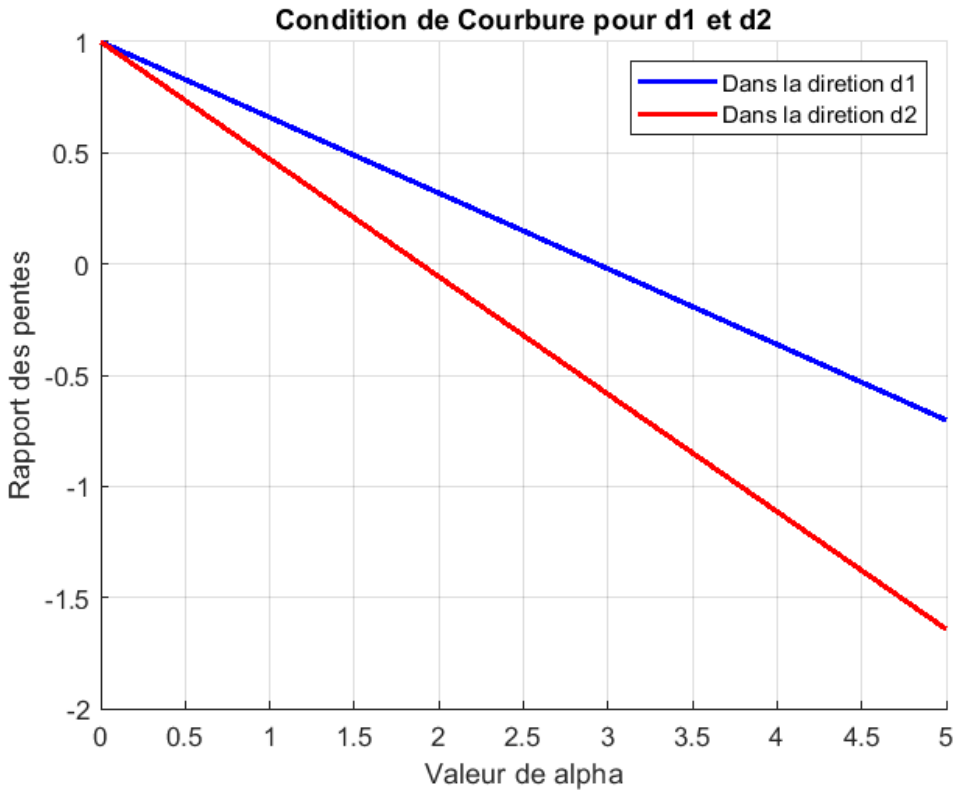


FIGURE 6 – Représentation de la condition de courbure

4.1.2 Justification numérique de la linéarité (direction h) du rapport des pentes

Soit la fonction $f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2$. Son gradient est donné par :

$$\nabla f_0(x) = \begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix} \tag{42}$$

Au point initial $x_0 = (10, 1)$, le gradient est $\nabla f_0(x_0) = (10, 9)^T$. Pour une direction de

recherche $h = (h_1, h_2)^T$, on définit le rapport des pentes $R(\alpha)$ par :

$$R(\alpha) = \frac{\nabla f_0(x_0 + \alpha h)^T h}{\nabla f_0(x_0)^T h} \tag{43}$$

Le développement numérique donne :

— **Dénominateur :** $\nabla f_0(x_0)^T h = 10h_1 + 9h_2$

— **Numérateur :** $\nabla f_0(x_0 + \alpha h)^T h = (10 + \alpha h_1)h_1 + (9 + 9\alpha h_2)h_2 = (10h_1 + 9h_2) + \alpha(h_1^2 + 9h_2^2)$

En substituant ces expressions dans $R(\alpha)$:

$$R(\alpha) = 1 + \alpha \left(\frac{h_1^2 + 9h_2^2}{10h_1 + 9h_2} \right) \tag{44}$$

Cette équation étant de type affine ($R(\alpha) = m\alpha + 1$), elle confirme que le graphe du rapport des pentes est une **droite** dont l'ordonnée à l'origine est égale à 1.

4.2 Interprétation de la condition de courbure (Wolfe 2) en fonction des valeurs de β_2

Graphiquement, cela revient à chercher les valeurs de α pour lesquelles la droite $R(\alpha)$ passe sous l'horizontale d'ordonnée β_2.

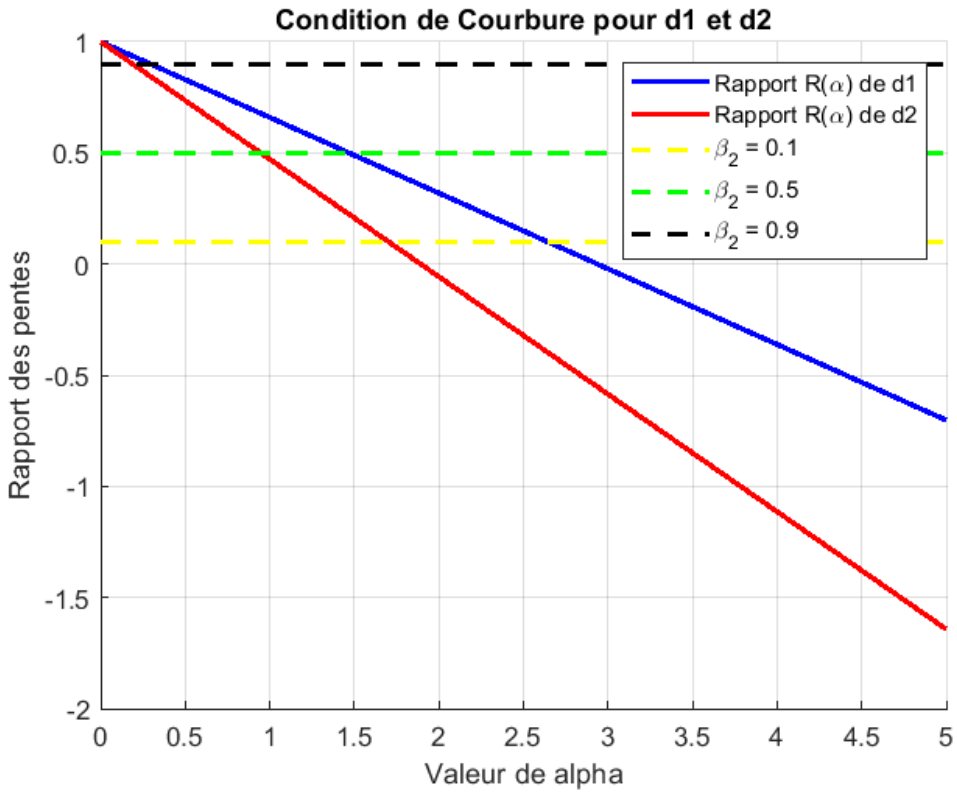


FIGURE 7 – Synthèse deuxieme condition de Wolfe : choix du meilleur pas

D'après la seconde condition de Wolfe (1), un pas α est acceptable si le rapport $R(\alpha)$ satisfait :

$$R(\alpha) = \frac{\nabla f_0(x_0 + \alpha h)^T h}{\nabla f_0(x_0)^T h} \leq \beta_2 \tag{45}$$

Puisque la direction h est une direction de descente, le dénominateur $\nabla f_0(x_0)^T d$ est strictement négatif. L'interprétation de ce seuil β_2 est la suivante :

Les figures ci-dessus permettent de comprendre le rôle de la deuxième condition de Wolfe :

$$\nabla f(x_0 + \alpha h)^T h \geq \beta_2 \nabla f(x_0)^T h.$$

Comme la direction h est une direction de descente, on a

$$\nabla f(x_0)^T h < 0.$$

On peut donc réécrire cette condition sous la forme

$$R(\alpha) = \frac{\nabla f_0(x_0 + \alpha h)^T h}{\nabla f_0(x_0)^T h} \leq \beta_2.$$

La droite horizontale $y = \beta_2$ sert alors de ligne de comparaison :

— Si la courbe du rapport des pentes $R(\alpha)$ reste en dessous de cette droite, alors le pas α est acceptable ;

— Si elle reste au-dessus, cela signifie que la pente n'a pas encore suffisamment changé et que le pas est trop petit.

• Lorsque $\beta_2 = 0.9$, le seuil est très haut. La condition est donc très souple : il suffit que la pente se soit un peu redressée pour que le pas soit accepté. Ainsi, même des pas très courts peuvent être admissibles.

• Lorsque $\beta_2 = 0.1$, le seuil est beaucoup plus bas. La condition devient alors plus stricte : il faut que la pente ait fortement changé avant d'accepter le pas. Cela oblige l'algorithme à rejeter les petits pas.

• Lorsque $\beta_2 = 0.5$, on obtient un cas intermédiaire. La condition n'est ni trop sévère ni trop permissive. Elle constitue donc un compromis raisonnable entre stabilité et rapidité de progression.

Ainsi, le paramètre β_2 contrôle le niveau d'exigence sur l'évolution de la pente :

— Une petite valeur de β_2 rend la condition plus stricte ;

— Une valeur plus grande de β_2 rend la condition plus facile à satisfaire.

5 Exercice 15 : Programmation de la recherche linéaire de Wolfe

5.1 Principe général de la recherche linéaire de Wolfe

La recherche linéaire de Wolfe [Wol69, Wol71, NW06] consiste à choisir un pas α qui satisfait simultanément deux conditions :

— La **première condition de Wolfe**, qui impose une diminution suffisante de la fonction ;

— La **deuxième condition de Wolfe**, qui impose que la pente ait suffisamment évolué.

Cela permet d'éviter deux comportements extrêmes :

- Choisir un pas trop grand, qui dégrade la décroissance de la fonction ;
- Choisir un pas trop petit, qui ferait avancer l'algorithme trop lentement.

5.2 Programmation de la fonction `wolfe.m`

On programme, dans un fichier `wolfe.m`, une fonction Matlab de la forme

```
function [xnv,nbsim] = wolfe(sim,xk,alpha,dk,beta1,beta2)
```

Dans cette écriture :

— `sim` est le nom du simulateur ;

— `xk` est l'itéré courant ;

— `alpha` est la valeur initiale du pas ;

— `dk` est la direction de descente courante ;

— `beta1` et `beta2` sont les paramètres des conditions de Wolfe ;

— `xnv` est le nouveau point $x + \alpha d$ obtenu avec le pas retenu ;

— `nbsim` est le nombre d'appels au simulateur effectués pendant la recherche.

Le principe de l'algorithme est le suivant :

— si le pas testé est trop grand, on le réduit ;

— s'il est trop petit, on l'augmente ;

— dès que les deux conditions de Wolfe sont satisfaites, on l'accepte.

Le code Matlab correspondant est :

```
function [xnv,nbsim] = wolfe(sim,xk,alpha,dk,beta1,beta2)
nbsim = 0;
amax = 100;
amin = 0;
[fk,gk] = feval(sim,xk);
```

```
nbsim = nbsim + 1;
gtd_initial = gk' * dk;

while (amax - amin) > 1e-10
xtest = xk + alpha*dk;
[ftest,gtest] = feval(sim,xtest);
nbsim = nbsim + 1;

% test de la premiere condition de Wolfe
if ftest > fk + beta1*alpha*gtd_initial
amax = alpha;
alpha = (amin + amax)/2;
else
% test de la deuxieme condition de Wolfe
gtd_test = gtest' * dk;
if gtd_test < beta2*gtd_initial
amin = alpha;
if amax == 100
alpha = 2*alpha;
else
alpha = (amin + amax)/2;
end
else
break;
end
end
end

xnv = xk + alpha*dk;
end
```

5.3 Intégration de Wolfe dans l'algorithme de descente : fichier `gradrl.m`

5.3.1 Script Matlab `gradrl.m`

```
function [x_k, k, X, nbsim_total] = gradrl(x_0, epsrel)

x_k = x_0;
```

```
k = 0;
X = x_k;

beta1 = 0.1;
beta2 = 0.9;
f0_sim = 'simf0';

[~, g] = simf0(x_k);
g0 = norm(g);
nbsim_total = 1;

while norm(g) > epsrel * g0
dk = -g;
[xnv, nbsim_wolfe] = wolfe(f0_sim, x_k, 1, dk, beta1, beta2);
x_k = xnv;
nbsim_total = nbsim_total + nbsim_wolfe;

[~, g] = simf0(x_k);
nbsim_total = nbsim_total + 1;

X = [X x_k];
k = k + 1;
end

end
```

5.4 Résultats numériques sur la fonction f_0

5.4.1 Approximation du minimum

On obtient numériquement un point très proche de l'origine :

```
>> [xk, k, X, nbsim_total] = gradrl([9;1], 1e-8);
>> disp(xk)
1.0e-07 *
0.5596
-0.0247
```

Le résultat fourni est de l'ordre de

$$x_k \approx \begin{pmatrix} 0.5596 \times 10^{-7} \\ -0.0247 \times 10^{-7} \end{pmatrix}, \tag{46}$$

ce qui est bien cohérent avec le minimum théorique $(0, 0)^T$.

5.4.2 Gain en nombre d'itérations

Le nombre d'itérations affiché est :

```
>> [xk, k, X, nbsim_total] = gradrl([9;1], 1e-8);
>> disp(k)
47
```

La méthode avec recherche linéaire de Wolfe converge donc en **47 itérations**, contre **83 itérations** pour l'algorithme pfd.m a l'exercice 12 . Il y a donc un gain clair en nombre d'itérations.

5.5 Étude théorique de la fonction f_1

On considère maintenant la fonction

$$f_1(x) = \frac{1}{2}x_1^2 + x_1 \cos(x_2).$$

5.5.1 Calcul du gradient

Le gradient de f_1 est donné par

$$\nabla f_1(x) = \begin{pmatrix} \dfrac{\partial f_1}{\partial x_1} \\ \dfrac{\partial f_1}{\partial x_2} \end{pmatrix}.$$

En dérivant par rapport à x_1, on obtient

$$\frac{\partial f_1}{\partial x_1} = x_1 + \cos(x_2).$$

En dérivant par rapport à x_2, on obtient

$$\frac{\partial f_1}{\partial x_2} = -x_1 \sin(x_2).$$

Ainsi,

$$\nabla f_1(x) = \begin{pmatrix} x_1 + \cos(x_2) \\ -x_1 \sin(x_2) \end{pmatrix}. \tag{47}$$

5.5.2 Résolution à la main

Pour déterminer les points critiques de f_1, on résout

$$\nabla f_1(x) = 0,$$

c'est-à-dire le système

$$\begin{cases} x_1 + \cos(x_2) = 0, \\ -x_1 \sin(x_2) = 0. \end{cases}$$

La deuxième équation donne

$$x_1 \sin(x_2) = 0.$$

Il y a donc deux possibilités :

Premier cas : $x_1 = 0$ Dans ce cas, la première équation devient

$$\cos(x_2) = 0,$$

d'où

$$x_2 = \frac{\pi}{2} + k\pi, \qquad k \in \mathbb{Z}.$$

On obtient donc des points critiques de la forme

$$\left(0, \frac{\pi}{2} + k\pi\right).$$

Deuxième cas : $\sin(x_2) = 0$ On a alors

$$x_2 = k\pi, \qquad k \in \mathbb{Z}.$$

La première équation devient

$$x_1 + \cos(k\pi) = 0.$$

Or

$$\cos(k\pi) = (-1)^k,$$

donc

$$x_1 = -(-1)^k.$$

On obtient ainsi les points critiques

$$(x_1, x_2) = (-1, (2m)\pi) \quad \text{ou} \quad (1, (2m+1)\pi), \qquad m \in \mathbb{Z}.$$

5.5.3 Valeur de la fonction aux points critiques

Pour les points

$$(-1, 2m\pi),$$

on a

$$f_1(-1, 2m\pi) = \frac{1}{2}(-1)^2 + (-1)\cos(2m\pi) = \frac{1}{2} - 1 = -\frac{1}{2}.$$

Pour les points

$$(1, (2m+1)\pi),$$

on a

$$f_1(1, (2m+1)\pi) = \frac{1}{2}(1)^2 + 1 \cdot \cos((2m+1)\pi) = \frac{1}{2} - 1 = -\frac{1}{2}.$$

Ainsi, ces deux familles de points donnent la même valeur

$$f_1 = -\frac{1}{2},$$

qui correspond aux minima de la fonction.

On retiendra donc que la fonction f_1 possède plusieurs minima, par exemple

$$(-1, 0), \qquad (1, \pi), \qquad (-1, 2\pi), \qquad (1, 3\pi), \ldots$$

5.6 Résultats numériques sur la fonction f_1

On applique maintenant le même procédé à la fonction

$$f_1(x) = \frac{1}{2}x_1^2 + x_1\cos(x_2).$$

5.6.1 Approximation du minimum

En lançant l'algorithme avec le point initial $x_0 = (9, 1)^T$, on obtient numériquement :

```
>> [xk, k, X, nbsim_total] = gradrl_f1([9;1], 1e-8);
>> disp(xk)
1.0000
9.4248
```

Le point calculé est donc très proche de

$$x_k \approx \begin{pmatrix} 1 \\ 9.4248 \end{pmatrix} = \begin{pmatrix} 1 \\ 3\pi \end{pmatrix}.$$

Ce résultat est cohérent avec l'étude théorique précédente, puisque

$$(1, 3\pi)$$

est bien l'un des minima de la fonction f_1.

5.6.2 Nombre d'itérations

Le nombre d'itérations obtenu est :

```
>> disp(k)
8
```

La méthode converge en **8 itérations**.

5.6.3 Nombre total d'appels au simulateur

Le nombre total d'appels au simulateur est :

```
>> disp(nbsim_total)
25
```

On observe donc que la recherche linéaire de Wolfe reste plus coûteuse qu'un simple calcul explicite du pas, cependant elle conduit à une convergence rapide.

5.6.4 Interprétation des résultats

Cet essai montre que la méthode avec recherche linéaire de Wolfe s'adapte aussi à une fonction non quadratique comme f_1.

L'algorithme converge vers l'un des minima de la fonction avec un nombre d'itérations réduit. Cela confirme l'intérêt de la recherche linéaire : même lorsque l'on ne dispose plus d'une formule explicite du pas optimal, on peut tout de même choisir un pas efficace grâce aux conditions de Wolfe.

6 Exercice 16 : Implémentation de la méthode de Newton locale

6.1 Rappel sur la méthode de Newton locale

La méthode de Newton locale est une méthode itérative utilisée pour approcher un minimum d'une fonction différentiable [NW06, DS96, Kan48]. À chaque itération, la direction de Newton d_k est définie par la relation

$$H(x_k)d_k = -\nabla f(x_k),$$

où $H(x_k)$ désigne la matrice hessienne de f au point x_k.

Lorsque la hessienne est inversible, cette relation s'écrit aussi

$$d_k = -H(x_k)^{-1}\nabla f(x_k).$$

La mise à jour de l'itéré est alors donnée par

$$x_{k+1} = x_k + \alpha_k d_k,$$

où α_k est un pas choisi ici à l'aide d'une stratégie adaptative de type Wolfe.

L'intérêt principal de la méthode de Newton est qu'elle utilise à la fois le gradient et la hessienne. Elle prend donc en compte non seulement la pente de la fonction, mais aussi sa courbure locale.Dans des conditions favorables, cela permet d'obtenir une convergence quadratique, donc beaucoup plus rapide que celle de la descente de gradient classique [NW06, DS96].

6.2 Données du problème

Dans cet exercice, on applique la méthode de Newton locale à deux fonctions.

On commence d'abord par la fonction quadratique

$$f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2,$$

puis on considère ensuite la fonction non quadratique

$$f_1(x) = \frac{1}{2}x_1^2 + x_1\cos(x_2).$$

6.3 Modification du simulateur

La méthode de Newton nécessite la connaissance de la hessienne. On construit donc une nouvelle version du simulateur associé à f_0, notée `simf0_2.m`, qui retourne la valeur de la fonction, son gradient et sa hessienne.

6.3.1 Calcul du gradient et de la hessienne de f_0

Pour

$$f_0(x) = \frac{1}{2}x_1^2 + \frac{9}{2}x_2^2,$$

on a

$$\nabla f_0(x) = \begin{pmatrix} x_1 \\ 9x_2 \end{pmatrix}$$

et

$$H_0(x) = \begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix}.$$

On remarque que la hessienne est constante et symétrique définie positive.

6.3.2 Fonction `simf0_2.m`

```
function [f,g,h] = simf0_2(x)
% x est un vecteur colonne [x1; x2]

f = 0.5*x(1)^2 + 4.5*x(2)^2;
g = [x(1); 9*x(2)];
h = [1 0; 0 9];

end
```

6.4 Implémentation de la méthode de Newton locale

On programme ensuite l'algorithme dans un fichier `newton_loc.m`. À chaque itération :

— on calcule la direction de Newton en résolvant

$$H(x_k)d_k = -\nabla f(x_k),$$

— puis on choisit un pas α_k à l'aide d'une stratégie adaptative ;

— enfin on met à jour l'itéré.

6.4.1 Principe de l'algorithme

Lorsque la hessienne est définie positive, la direction de Newton est donnée par

$$d_k = -H(x_k)^{-1}\nabla f(x_k).$$

Dans le cas contraire, la direction de Newton peut ne plus être une direction de descente. Il est donc prudent, dans une implémentation numérique, de prévoir une stratégie de secours, par exemple en revenant à la direction du gradient opposé.

6.4.2 Fichier `newton_loc.m`

```
function [xk, fk, iter, X] = newton_loc(sim, x0, tol, maxit)

% xk represente le point courant, initialise avec le point de depart
xk = x0;

% X permettra de conserver la trace de tous les iteres successifs
X = xk;

% boucle principale de l'algorithme de Newton
for k = 1:maxit

% calcul de la fonction, du gradient et de la hessienne au point courant
[f, g, h] = feval(sim, xk);

% condition d'arret : on s'interrompt si le gradient devient tres petit
if norm(g) < tol
break;
end

% determination de la direction de descente
% si la hessienne est definie positive, on utilise la direction de Newton
if all(eig(h) > 0)
d = -h \ g;
else
% sinon, on se rabat sur une direction plus simple : l'oppose du gradient
d = -g;
end
```

```
% selection du pas avec un backtracking base sur la condition d'Armijo
alpha = 1;      % on essaie d'abord un pas complet
c1 = 1e-4;      % constante de la condition d'Armijo
rho = 0.5;      % coefficient de reduction du pas

while true
% nouveau point teste avec le pas actuel
xnew = xk + alpha*d;

% evaluation de la fonction en ce nouveau point
[fnew, ~, ~] = feval(sim, xnew);

% verification de la condition d'Armijo
% si elle est respectee, le pas est valide
if fnew <= f + c1*alpha*(g'*d)
break;
end

% sinon, on diminue le pas et on recommence
alpha = rho*alpha;
end

% mise a jour du point courant
xk = xnew;

% ajout du nouvel itere a l'historique
X = [X xk];
end

% valeur finale de la fonction
fk = f;

% nombre total d'iterations effectuees
iter = k;

end
```

6.5 Tests numériques sur la fonction f_0

On teste d'abord la méthode de Newton sur la fonction quadratique f_0.

```
>>   x0 = [9; 1];
[xk, fk, iter, X] = newton_loc('simf0_2', x0, 1e-8, 50);

disp('Point final :')
disp(xk)

disp('Nombre d''iterations :')
disp(iter)
Point final :
0
0

Nombre d'iterations :
2
```

6.5.1 Interprétation

Dans ce cas, la méthode converge très rapidement vers le minimum

$$x^* = (0, 0)^T.$$

Cela s'explique par le fait que la fonction f_0 est quadratique et que sa hessienne est constante. La méthode de Newton est alors particulièrement adaptée à ce type de fonction, ce qui explique sa grande efficacité.

6.6 Étude théorique de la fonction f_1

On considère maintenant la fonction

$$f_1(x) = \frac{1}{2}x_1^2 + x_1 \cos(x_2).$$

6.6.1 Calcul du gradient

Le gradient de f_1 est

$$\nabla f_1(x) = \begin{pmatrix} x_1 + \cos(x_2) \\ -x_1 \sin(x_2) \end{pmatrix}.$$

6.6.2 Calcul de la hessienne

La hessienne de f_1 vaut

$$H_1(x) = \begin{pmatrix} 1 & -\sin(x_2) \\ -\sin(x_2) & -x_1\cos(x_2) \end{pmatrix}.$$

6.6.3 Résolution à la main des points critiques

Les points critiques sont obtenus en résolvant

$$\nabla f_1(x) = 0,$$

c'est-à-dire

$$\begin{cases} x_1 + \cos(x_2) = 0, \\ -x_1\sin(x_2) = 0. \end{cases}$$

La deuxième équation donne

$$x_1\sin(x_2) = 0.$$

On distingue alors deux cas :

Cas 1 : $x_1 = 0$ La première équation devient

$$\cos(x_2) = 0,$$

donc

$$x_2 = \frac{\pi}{2} + k\pi, \qquad k \in \mathbb{Z}.$$

On obtient ainsi des points critiques de la forme

$$\left(0, \frac{\pi}{2} + k\pi\right).$$

Cas 2 : $\sin(x_2) = 0$ On a alors

$$x_2 = k\pi, \qquad k \in \mathbb{Z},$$

et la première équation devient

$$x_1 + \cos(k\pi) = 0.$$

Comme

$$\cos(k\pi) = (-1)^k,$$

on obtient

$$x_1 = -(-1)^k.$$

Donc les autres points critiques sont

$$(-1, 2m\pi) \quad \text{et} \quad (1, (2m+1)\pi), \qquad m \in \mathbb{Z}.$$

6.6.4 Quels sont les minima ?

On calcule la valeur de f_1 sur ces deux familles :

$$f_1(-1, 2m\pi) = \frac{1}{2}(-1)^2 + (-1)\cos(2m\pi) = \frac{1}{2} - 1 = -\frac{1}{2},$$

et

$$f_1(1, (2m+1)\pi) = \frac{1}{2}(1)^2 + 1 \cdot \cos((2m+1)\pi) = \frac{1}{2} - 1 = -\frac{1}{2}.$$

Ces deux familles correspondent donc aux minima de f_1.

On peut retenir par exemple les minima suivants :

$$(-1, 0), \qquad (1, \pi), \qquad (-1, 2\pi), \qquad (1, 3\pi), \ldots$$

6.6.5 Fonction `simf1_2.m`

```
function [f,g,h] = simf1_2(x)
% x est un vecteur colonne [x1; x2]

f = 0.5*x(1)^2 + x(1)*cos(x(2));

g = [x(1) + cos(x(2));
-x(1)*sin(x(2))];

h = [1, -sin(x(2));
-sin(x(2)), -x(1)*cos(x(2))];

end
```

6.7 Tests numériques sur la fonction f_1

6.7.1 Résultats des tests de la méthode de Newton

Test 1

k	$x_1^{(k)}$	$x_2^{(k)}$
0	-0.9000	0.4000
1	-1.1273	-0.1296
2	-1.0067	-0.0132
3	-1.0001	-0.0001
4	-1.0000	-0.0000

Test 2

k	$x_1^{(k)}$	$x_2^{(k)}$
0	-1.2000	0.2000
1	-1.0141	0.0287
2	-1.0004	0.0004
3	-1.0000	0.0000
4	-1.0000	0.0000

Test 3

k	$x_1^{(k)}$	$x_2^{(k)}$
0	0.5000	2.5000
1	0.9418	3.2067
2	1.0024	3.1373
3	1.0000	3.1416
4	1.0000	3.1416

Test 4

k	$x_1^{(k)}$	$x_2^{(k)}$
0	2.0000	1.0000
1	-0.5403	2.6829
2	0.8967	2.4437
3	1.4085	3.3418
4	1.0089	3.1964
5	1.0015	3.1419
6	1.0000	3.1416
7	1.0000	3.1416

Test 5

k	$x_1^{(k)}$	$x_2^{(k)}$
0	-2.0000	3.0000
1	0.9900	2.7178
2	1.1218	3.2290
3	1.0030	3.1506
4	1.0000	3.1416
5	1.0000	3.1416

Test 6

k	$x_1^{(k)}$	$x_2^{(k)}$
0	3.0000	-1.0000
1	-0.5403	-3.5244
2	0.9276	-3.7262
3	1.1516	-3.2355
4	1.0033	-3.1534
5	1.0001	-3.1416
6	1.0000	-3.1416

Tableau récapitulatif final

$x_{0,1}$	$x_{0,2}$	$x_{f,1}$	$x_{f,2}$	f_{final}	Itérations	Convergence
-0.9	0.4	-1	-7.3573×10^{-9}	-0.5	5	oui
-1.2	0.2	-1	1.0962×10^{-14}	-0.5	5	oui
0.5	2.5	1	3.1416	-0.5	5	oui
2	1	1	3.1416	-0.5	8	oui
-2	3	1	3.1416	-0.5	6	oui
3	-1	1	-3.1416	-0.5	7	oui

Signification des notations

Dans les tableaux précédents, les différentes notations ont la signification suivante. Pour chaque test, on part d'un point initial

$$x_0 = (x_{0,1}, x_{0,2})^T.$$

À l'itération k, la méthode de Newton fournit un point

$$x^{(k)} = \left(x_1^{(k)}, x_2^{(k)}\right)^T,$$

où :

$x_1^{(k)}$ est la première composante de l'itéré $x^{(k)}$,

$x_2^{(k)}$ est la deuxième composante de l'itéré $x^{(k)}$.

Dans les tableaux des tests :

Terme	Signification
k	numéro de l'itération
$x_1^{(k)}$	première coordonnée du point calculé à l'itération k
$x_2^{(k)}$	deuxième coordonnée du point calculé à l'itération k

Dans le tableau récapitulatif final :

Terme	Signification
$x_{0,1}$	première coordonnée du point de départ x_0
$x_{0,2}$	deuxième coordonnée du point de départ x_0
$x_{f,1}$	première coordonnée du point final obtenu
$x_{f,2}$	deuxième coordonnée du point final obtenu
f_{final}	valeur de la fonction au point final, c'est-à-dire $f(x_f)$
Itérations	nombre total d'itérations effectuées par la méthode
Convergence	indique si la méthode a convergé ou non

Ici, le point final

$$x_f = (x_{f,1}, x_{f,2})^T$$

correspond à l'approximation numérique du minimum obtenue par la méthode de Newton.

La mention *oui* dans la colonne *Convergence* signifie que le critère d'arrêt a été satisfait, c'est-à-dire que la norme du gradient au point final est devenue inférieure à la tolérance fixée.

6.7.2 Interprétation

Les résultats montrent que la méthode de Newton est très efficace lorsque le point de départ est proche d'un minimum. Dans ce cas, elle converge en très peu d'itérations vers la solution.

Lorsque le point initial est plus éloigné, la méthode converge encore dans les tests effectués, mais elle peut demander davantage d'itérations. On remarque aussi que le minimum obtenu dépend du point de départ choisi.

Cela confirme que la méthode de Newton est une méthode locale : elle est très rapide près d'un minimum, mais son comportement global reste sensible à l'initialisation.

7 Exercice 17 : Comparaison de la méthode de gradient et de la méthode de Newton locale

7.1 Étude théorique des fonctions

7.1.1 Fonction f_2

On considère

$$f_2(x) = x_1^4 + x_2^4.$$

Son gradient est

$$\nabla f_2(x) = \begin{pmatrix} 4x_1^3 \\ 4x_2^3 \end{pmatrix}.$$

Sa hessienne est

$$H_2(x) = \begin{pmatrix} 12x_1^2 & 0 \\ 0 & 12x_2^2 \end{pmatrix}.$$

On remarque que f_2 est une fonction convexe, de minimum global

$$x^* = (0,0)^T, \qquad f_2(x^*) = 0.$$

7.1.2 Fonction f_3

On considère maintenant

$$f_3(x) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2.$$

Son gradient est

$$\nabla f_3(x) = \begin{pmatrix} -2(1 - x_1) - 400x_1(x_2 - x_1^2) \\ 200(x_2 - x_1^2) \end{pmatrix}.$$

Sa hessienne est

$$H_3(x) = \begin{pmatrix} 2 - 400x_2 + 1200x_1^2 & -400x_1 \\ -400x_1 & 200 \end{pmatrix}.$$

La fonction f_3 admet un minimum global en

$$x^* = (1,1)^T, \qquad f_3(x^*) = 0.$$

Cette fonction est plus délicate que f_2, car sa vallée de minimum est étroite. Cela rend souvent la descente de gradient plus lente, tandis que Newton peut devenir très efficace lorsque l'on est suffisamment proche du minimum.

7.2 Simulateurs Matlab

Pour comparer les deux méthodes, on construit des simulateurs renvoyant la fonction, son gradient, et éventuellement sa hessienne.

7.2.1 Fonction `simf2_2.m`

```
function [f,g,h] = simf2_2(x)

f = x(1)^4 + x(2)^4;

g = [4*x(1)^3;
```

```
4*x(2)^3];

h = [12*x(1)^2, 0;
0, 12*x(2)^2];

end
```

7.2.2 Fonction `simf3_2.m`

```
function [f,g,h] = simf3_2(x)

f = (1-x(1))^2 + 100*(x(2)-x(1)^2)^2;

g = [-2*(1-x(1)) - 400*x(1)*(x(2)-x(1)^2);
200*(x(2)-x(1)^2)];

h = [2 - 400*x(2) + 1200*x(1)^2, -400*x(1);
-400*x(1),                   200];

end
```

7.3 Principe de la comparaison

Pour chaque fonction, on applique :

— la méthode du gradient avec pas adaptatif ;

— la méthode de Newton locale avec pas adaptatif.

On compare ensuite :

— la trajectoire des itérés sur la carte de niveaux ;

— l'évolution de $f(x_k)$;

— l'évolution de $\|\nabla f(x_k)\|$.

7.4 Tests numériques sur la fonction f_2

Données de lancement et implémentation

```
x0 = [1.5; -1.5];

[xg, kg, Xg, nbsim_g] = gradrl_gen('simf2', x0, 1e-8);
[xn, fn, kn, Xn] = newton_loc('simf2_2', x0, 1e-8, 100);
```

7.4.1 Carte de niveaux et trajectoires

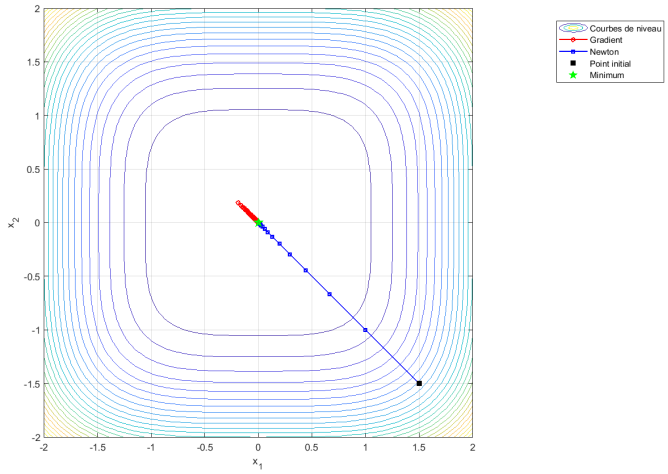


FIGURE 8 – Comparaison des trajectoires pour f_2

7.4.2 Historique de convergence

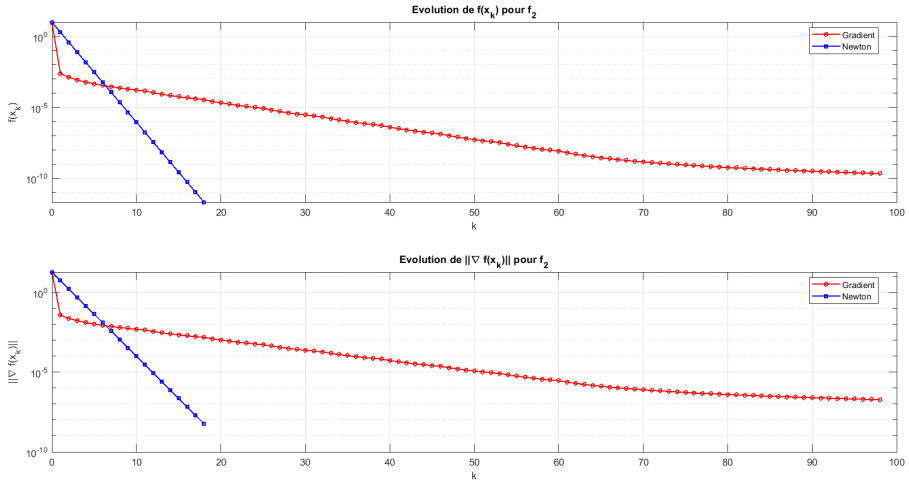


FIGURE 9 – Evolution de $||\nabla f(x_k)||$ et de $f(x_k)$ pour f_2 en fonction des itérations k

7.4.3 Interprétation

Pour la fonction f_2, les deux méthodes convergent bien vers le minimum $(0, 0)^T$. Cependant, la méthode de Newton atteint ce minimum plus rapidement, car elle exploite la courbure locale (grace à la hessienne) de la fonction en plus du gradient [NW06, DS96].

La méthode du gradient reste plus simple à mettre en œuvre, mais elle avance de manière plus progressive et demande en général davantage d'itérations pour converger

[Cau47, NW06].

7.5 Tests numériques sur la fonction f_3

Données de lancement

```
x0 = [-1.5; -1.5];

[xg, kg, Xg, nbsim_g] = gradrl_gen('simf3', x0, 1e-8);
[xn, fn, kn, Xn] = newton_loc('simf3_2', x0, 1e-8, 100);
```

7.5.1 Carte de niveaux et trajectoires

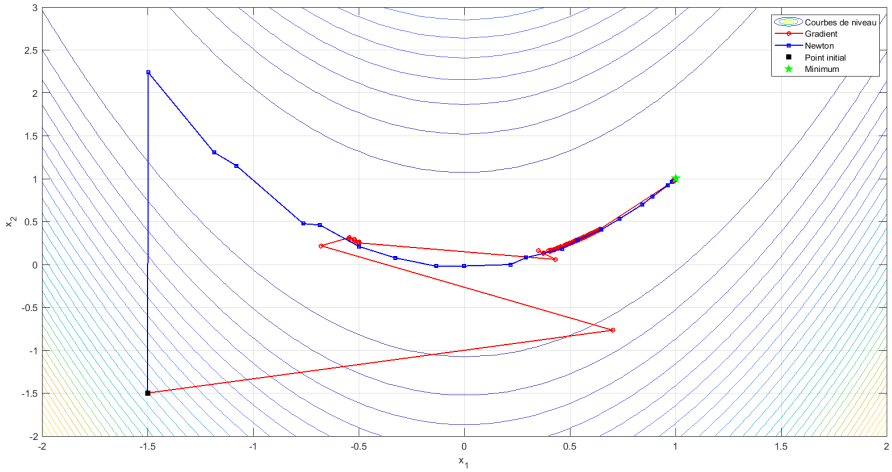


FIGURE 10 – Comparaison des trajectoires pour f_3

7.5.2 Historique de convergence

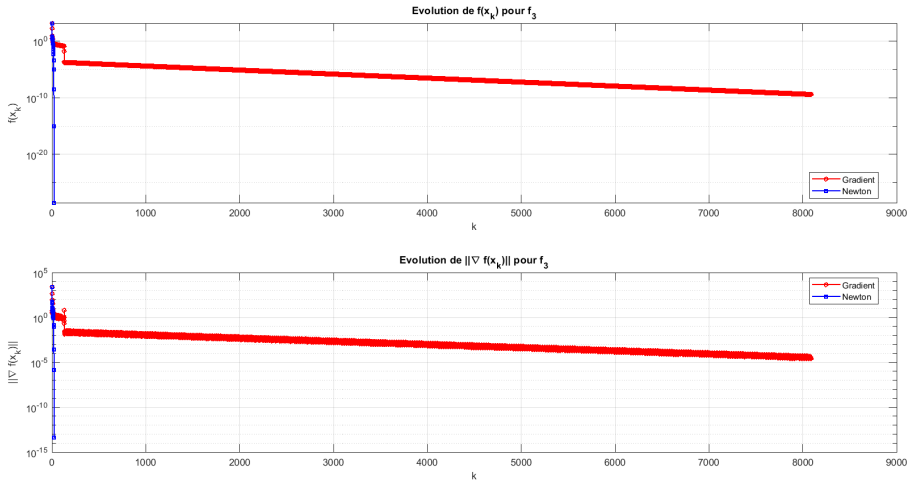


FIGURE 11 – Evolution de $||\nabla f(x_k)||$ et de $f(x_k)$ pour f_3 en fonction des itérations k

7.5.3 Interprétation

Pour la fonction f_3, on remarque que la méthode du gradient progresse plus lentement. Les itérés ne vont pas directement vers le minimum $(1,1)^T$, mais avancent de manière irrégulière, avec une trajectoire souvent en zigzag. Cela montre que, pour cette fonction, la simple direction du gradient ne permet pas toujours d'aller rapidement vers la solution.

La méthode de Newton, en revanche, devient plus efficace lorsque le point de départ est assez proche du minimum. Elle converge alors plus rapidement, car elle utilise aussi les informations données par la hessienne. En revanche, cette méthode reste plus sensible au point initial : si l'on part trop loin, ou si la hessienne n'est pas favorable, la convergence peut être moins bonne.

7.6 Synthèse

Cette comparaison met en évidence les différences essentielles entre les deux méthodes.

La méthode du gradient est plus simple et plus robuste, mais elle peut être lente.

La méthode de Newton locale est plus rapide lorsqu'elle fonctionne dans de bonnes conditions, car elle exploite la courbure de la fonction. En revanche, elle dépend davantage de la qualité du point de départ et de la hessienne.

On peut donc conclure que la méthode de Newton est souvent plus performante près du minimum, tandis que la méthode du gradient reste plus simple à mettre en œuvre et plus stable globalement.

8 Exercice 18 : Ajustement d'une gaussienne par la méthode de Gauss-Newton

8.1 Situation du problème

Soit un nuage de points

$$(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n),$$

On cherche une courbe de type gaussienne qui passe le plus près possible de ces points.

La courbe choisie est de la forme suivante :

$$g(x) = \alpha_1 + \alpha_2 \exp\big(-\alpha_4(x - \alpha_3)^2\big), \tag{48}$$

où

$$a = (\alpha_1, \alpha_2, \alpha_3, \alpha_4)^T \in \mathbb{R}^4$$

est le vecteur des paramètres inconnus avec des significations suivantes :

- α_1 correspond au niveau de base de la courbe ;
- α_2 correspond à la hauteur de la bosse ;
- α_3 correspond à la position du centre de la gaussienne ;
- α_4 contrôle l'étalement de la courbe.

8.2 Fonction coût à minimiser

L'erreur sur chaque point est donnée par :

$$g(x_i) - y_i.$$

Comme en méthode des moindres carrés, on additionne ensuite les carrés de ces erreurs. On définit alors la fonction coût

$$f(\alpha_1, \alpha_2, \alpha_3, \alpha_4) = \frac{1}{2} \sum_{i=1}^{n} \big(\alpha_1 + \alpha_2 \exp\big(-\alpha_4(x_i - \alpha_3)^2\big) - y_i\big)^2. \tag{49}$$

Le but de l'exercice est donc de résoudre

$$\min_{a \in \mathbb{R}^4} f(a).$$

8.3 Écriture de la fonction sous la forme $f(a) = \frac{1}{2}\|h(a)\|^2$

Pour appliquer la méthode de Gauss-Newton, on peut écrire le problème sous une forme adaptée.

On commence par définir les **résidus**. Un résidu est l'erreur entre la valeur donnée par la méthode et la donnée expérimentale.

Pour chaque point i, on pose

$$h_i(a) = \alpha_1 + \alpha_2 \exp\big(-\alpha_4(x_i - \alpha_3)^2\big) - y_i. \tag{50}$$

On regroupe ensuite tous ces résidus dans un vecteur

$$h(a) = \begin{pmatrix} h_1(a) \\ h_2(a) \\ \vdots \\ h_n(a) \end{pmatrix} \in \mathbb{R}^n. \tag{51}$$

Ainsi, la fonction coût devient

$$f(a) = \frac{1}{2}\|h(a)\|^2. \tag{52}$$

Ainsi le vecteur $h(a)$ contient toutes les erreurs de la méthode. Donc minimiser $f(a)$ revient à rendre ce vecteur d'erreurs aussi petit que possible ce qui est le but recherché.

NB : attention ici pour avoir une courbe gaussienne le coefficient α_4 doit être strictement positif. Il faudrait en toute rigueur faire une optimisation sous contraintes, mais nous nous contenterons de vérifier que cette contrainte est satisfaite.

8.4 Expression explicite de la fonction $h(a)$

Si

$$a = (\alpha_1, \alpha_2, \alpha_3, \alpha_4)^T,$$

alors

$$h(a) = \begin{pmatrix} \alpha_1 + \alpha_2 e^{-\alpha_4(x_1-\alpha_3)^2} - y_1 \\ \alpha_1 + \alpha_2 e^{-\alpha_4(x_2-\alpha_3)^2} - y_2 \\ \vdots \\ \alpha_1 + \alpha_2 e^{-\alpha_4(x_n-\alpha_3)^2} - y_n \end{pmatrix}. \tag{53}$$

8.4.1 Fonction `calculeh.m`

```
function h = calculeh(X, a)
% X : matrice 2 x n, chaque colonne est [x_i ; y_i]
```

```
% a : vecteur colonne [a1; a2; a3; a4]

x = X(1,:)';
y = X(2,:)';

a1 = a(1);
a2 = a(2);
a3 = a(3);
a4 = a(4);

h = a1 + a2*exp(-a4*(x-a3).^2) - y;
end
```

8.5 Calcul de la matrice Jacobienne $Dh(a)$

La méthode de Gauss-Newton [Bjö96, Gau09, NW06] utilise la dérivée de $h(a)$. Comme $h(a)$ est un vecteur, sa dérivée est une matrice : c'est la **Jacobienne**.

Cette matrice contient toutes les dérivées partielles des composantes $h_i(a)$ par rapport aux paramètres $\alpha_1, \alpha_2, \alpha_3, \alpha_4$.

8.5.1 Résidu élémentaire

Pour chaque i, on a

$$h_i(a) = \alpha_1 + \alpha_2 e^{-\alpha_4(x_i-\alpha_3)^2} - y_i. \tag{54}$$

Pour simplifier l'écriture, on pose :

$$E_i = e^{-\alpha_4(x_i-\alpha_3)^2}. \tag{55}$$

Donc

$$h_i(a) = \alpha_1 + \alpha_2 E_i - y_i. \tag{56}$$

8.5.2 Dérivée par rapport à α_1

$$\frac{\partial h_i}{\partial \alpha_1} = 1. \tag{57}$$

8.5.3 Dérivée par rapport à α_2

$$\frac{\partial h_i}{\partial \alpha_2} = E_i = e^{-\alpha_4(x_i-\alpha_3)^2}. \tag{58}$$

8.5.4 Dérivée par rapport à α_3

On a

$$\frac{\partial E_i}{\partial \alpha_3} = e^{-\alpha_4(x_i-\alpha_3)^2} \cdot \frac{\partial}{\partial \alpha_3}\big(-\alpha_4(x_i-\alpha_3)^2\big). \tag{59}$$

Or

$$\frac{\partial}{\partial \alpha_3}(x_i-\alpha_3)^2 = -2(x_i-\alpha_3). \tag{60}$$

Donc

$$\frac{\partial}{\partial \alpha_3}\big(-\alpha_4(x_i-\alpha_3)^2\big) = 2\alpha_4(x_i-\alpha_3). \tag{61}$$

Par conséquent,

$$\frac{\partial E_i}{\partial \alpha_3} = 2\alpha_4(x_i-\alpha_3)E_i. \tag{62}$$

Finalement,

$$\frac{\partial h_i}{\partial \alpha_3} = 2\alpha_2\alpha_4(x_i-\alpha_3)E_i. \tag{63}$$

8.5.5 Dérivée par rapport à α_4

De la même façon,

$$\frac{\partial E_i}{\partial \alpha_4} = e^{-\alpha_4(x_i-\alpha_3)^2} \cdot \frac{\partial}{\partial \alpha_4}\big(-\alpha_4(x_i-\alpha_3)^2\big). \tag{64}$$

Comme

$$\frac{\partial}{\partial \alpha_4}\big(-\alpha_4(x_i-\alpha_3)^2\big) = -(x_i-\alpha_3)^2, \tag{65}$$

on obtient

$$\frac{\partial E_i}{\partial \alpha_4} = -(x_i-\alpha_3)^2 E_i. \tag{66}$$

Ainsi,

$$\frac{\partial h_i}{\partial \alpha_4} = -\alpha_2(x_i-\alpha_3)^2 E_i. \tag{67}$$

8.5.6 Forme finale de la Jacobienne

La i-ième ligne de la matrice Jacobienne est donc

$$\big(1,\ E_i,\ 2\alpha_2\alpha_4(x_i-\alpha_3)E_i,\ -\alpha_2(x_i-\alpha_3)^2E_i\big)\,. \tag{68}$$

On en déduit

$$Dh(a) = \begin{pmatrix} 1 & E_1 & 2\alpha_2\alpha_4(x_1-\alpha_3)E_1 & -\alpha_2(x_1-\alpha_3)^2E_1 \\ 1 & E_2 & 2\alpha_2\alpha_4(x_2-\alpha_3)E_2 & -\alpha_2(x_2-\alpha_3)^2E_2 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & E_n & 2\alpha_2\alpha_4(x_n-\alpha_3)E_n & -\alpha_2(x_n-\alpha_3)^2E_n \end{pmatrix}. \tag{69}$$

Fonction `calculeDh.m`

```
function J = calculeDh(X, a)
% J : Jacobienne de h, de taille n x 4

x = X(1,:)';

a2 = a(2);
a3 = a(3);
a4 = a(4);

E = exp(-a4*(x-a3).^2);

J = zeros(length(x),4);

J(:,1) = 1;
J(:,2) = E;
J(:,3) = 2*a2*a4*(x-a3).*E;
J(:,4) = -a2*(x-a3).^2.*E;
end
```

8.6 Validation numérique de la Jacobienne

Afin de vérifier que la Jacobienne calculée est correcte, nous procédons à la comparaison entre

$$\frac{h(a+\varepsilon d)-h(a)}{\varepsilon} \qquad \text{et} \qquad Dh(a)d, \tag{70}$$

où $d \in \mathbb{R}^4$ est une direction fixée et ε est un petit nombre.

8.6.1 Interprétation

— Si l'erreur diminue quand ε diminue, la Jacobienne est probablement correcte.
— Si l'erreur reste grande, la Jacobienne est probablement incorrecte.

Script Matlab de validation

```
a = [0; 1; 0; 0.5];
d = randn(4,1);

eps_list = 10.^(-(1:12));
```

```
err = zeros(size(eps_list));

for k = 1:length(eps_list)
epsi = eps_list(k);

v1 = (calculeh(X, a + epsi*d) - calculeh(X, a))/epsi;
v2 = calculeDh(X, a)*d;

err(k) = norm(v1 - v2);
end

loglog(eps_list, err, 'o-');
grid on;
xlabel('\epsilon');
ylabel('Erreur');
title('Validation numérique de la Jacobienne');
```

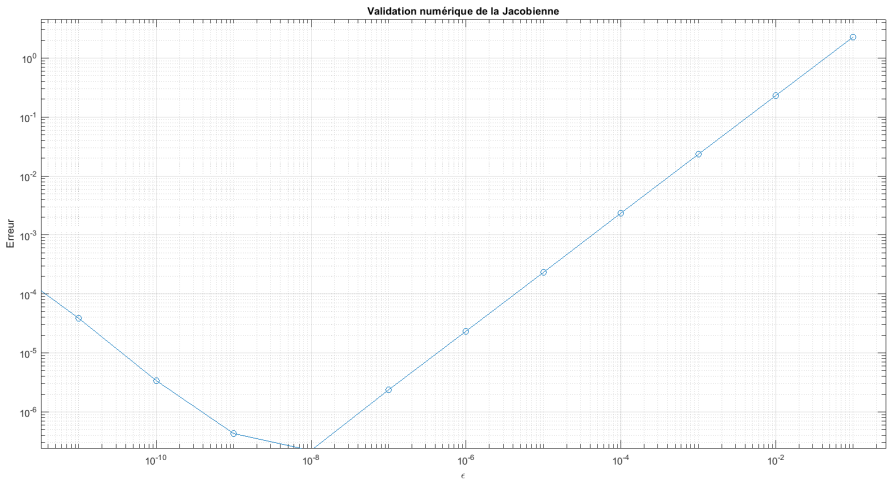


FIGURE 12 – Evolution de l'erreur en fonction de epsilon

Remarques

- Pour des ϵ pas trop petits, l'erreur décroît à peu près comme ϵ
- Si ϵ devient trop petit, les erreurs d'arrondi machine prennent le dessus et on observe une croissance au lieu d'une décroissance.

8.7 Implémentation Matlab du script principal

8.7.1 Rappel : Principe de la méthode de Gauss-Newton

La fonction coût s'écrit sous la forme

$$f(a) = \frac{1}{2}\|h(a)\|^2.$$

La méthode de Gauss-Newton [Bjö96, NW06] consiste à approcher localement h par une fonction affine.

Par une linéarisation (développement de Taylor au premier ordre) au voisinage d'un point a_k, on a

$$h(a_k + d) \approx h(a_k) + Dh(a_k)d. \tag{71}$$

En posant

$$h_k = h(a_k), \qquad J_k = Dh(a_k),$$

on remplace alors le problème initial par le problème approché

$$\min_{d\in\mathbb{R}^4} \frac{1}{2}\|h_k + J_k d\|^2. \tag{72}$$

Ce nouveau problème est un problème de moindres carrés linéaire.

En écrivant les équations normales, on obtient

$$J_k^T J_k d_k = -J_k^T h_k. \tag{73}$$

Donc la direction de Gauss-Newton est

$$d_k = -(J_k^T J_k)^{-1} J_k^T h_k, \tag{74}$$

lorsque la matrice $J_k^T J_k$ est inversible.

Comme indiqué dans l'énoncé, on peut utiliser une version simple de l'algorithme en prenant

$$a_{k+1} = a_k + \rho d_k, \tag{75}$$

avec par exemple

$$\rho = 0.1.$$

8.7.2 Script Mathlab principal

Fonction `interpogaussienne.m`

```
function a = interpogaussienne(X, a0)
```

```
a = a0(:);
maxit = 100;
tol = 1e-8;
rho = 0.1;

for k = 1:maxit
h = calculeh(X, a);
J = calculeDh(X, a);

% Direction de Gauss-Newton
d = -((J' * J) \ (J' * h));

% Mise a jour
anew = a + rho*d;

% Verification de a4
if anew(4) <= 0
warning('Le parametre a4 devient non positif.');
break;
end

% Critere d'arret
if norm(anew - a) <= tol*(1 + norm(a))
a = anew;
break;
end

a = anew;
end

% Trace du resultat
x = X(1,:);
y = X(2,:);

xx = linspace(min(x), max(x), 400);
yy = a(1) + a(2)*exp(-a(4)*(xx-a(3)).^2);

figure;
plot(x, y, 'bo', 'MarkerFaceColor', 'b');
```

```
hold on;
plot(xx, yy, 'r-', 'LineWidth', 2);
grid on;
xlabel('x');
ylabel('y');
legend('Donnees', 'Gaussienne ajustee');
title('Ajustement d''une gaussienne par Gauss-Newton');
end
```

8.8 Application : Test de la fonction interpogaussienne.m

Premier essai

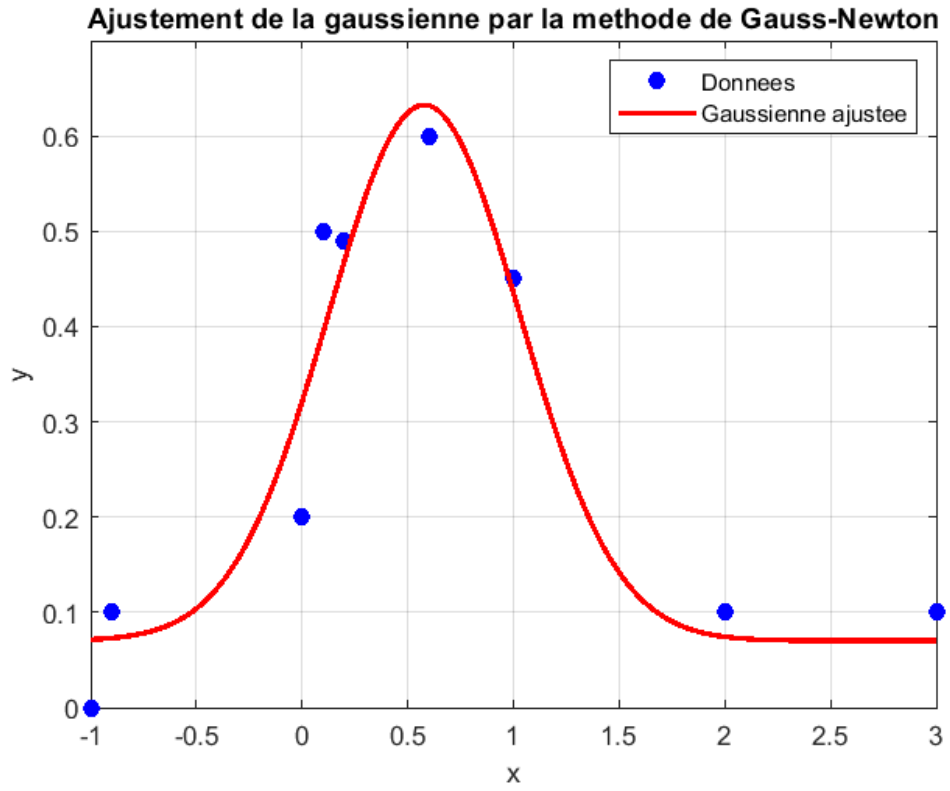


FIGURE 13 – Données entrées : $a_0 = [0; 1; 0; 0.5]$;
$X = \begin{bmatrix} -1 & -0.9 & 0 & 0.1 & 0.2 & 1 & 2 & 3 & 0.6 \\ 0 & 0.1 & 0.2 & 0.5 & 0.49 & 0.45 & 0.1 & 0.1 & 0.6 \end{bmatrix}$

Sortie :

```
a = [ 0.0702
0.5624
0.5775
```

```
2.4202 ]
```

Deuxième essai : Données avec bruit

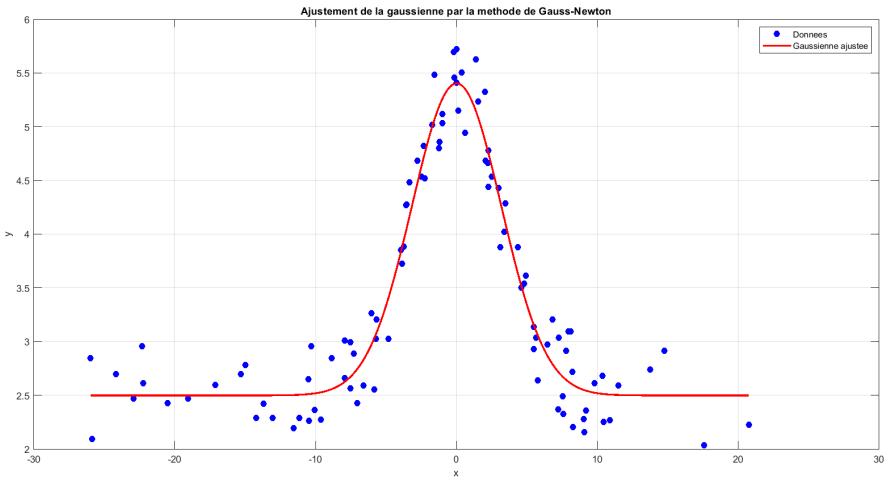


FIGURE 14 – Données d'entrée : `a0 = [0; 1; 0; 0.5]`.
Génération des échantillons : `X1 = 10*randn(1,100) - 1` et
`X = [X1 ; 2 + 3*exp(-0.05*X1.^2) + rand(1,100)]`.

Sortie :

```
a = [ 2.4983
2.9048
0.0231
0.0496 ]
```

9 Références théoriques

Les méthodes de recherche linéaire utilisées dans ce rapport s'appuient sur la règle de diminution suffisante d'Armijo [Arm66] et sur les conditions de Wolfe [Wol69, Wol71].

Pour une présentation moderne des méthodes du gradient, de la recherche linéaire et de la méthode de Newton locale, on pourra consulter [NW06, DS96].

La méthode de Gauss-Newton utilisée pour l'ajustement non linéaire s'inscrit dans le cadre des problèmes de moindres carrés non linéaires [Bjö96].

Références

[Arm66] Larry Armijo. Minimization of functions having lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1) :1–3, 1966.

[Bjö96] Åke Björck. *Numerical Methods for Least Squares Problems*. Society for Industrial and Applied Mathematics, Philadelphia, 1996.

[Cau47] Augustin-Louis Cauchy. Méthode générale pour la résolution des systèmes d'équations simultanées. *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences*, 25 :536–538, 1847.

[DS96] J. E. Dennis, Jr. and Robert B. Schnabel. *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, volume 16 of *Classics in Applied Mathematics*. Society for Industrial and Applied Mathematics, Philadelphia, 1996.

[Gau09] Carl Friedrich Gauss. *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*. Sumtibus F. Perthes et I. H. Besser, Hamburg, 1809.

[Kan48] L. V. Kantorovich. On newton's method for functional equations. *Doklady Akademii Nauk SSSR*, 59 :1237–1240, 1948. In Russian.

[NW06] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, 2nd edition, 2006.

[Wol69] Philip Wolfe. Convergence conditions for ascent methods. *SIAM Review*, 11(2) :226–235, 1969.

[Wol71] Philip Wolfe. Convergence conditions for ascent methods. ii : Some corrections. *SIAM Review*, 13(2) :185–188, 1971.